

Low Precision Floating-point Arithmetic for High Performance FPGA-based CNN Acceleration

Chen Wu, Mingyu Wang, Xinyuan Chu, Kun Wang, Lei He

Abstract—Low precision data representation is important to reduce storage size and memory access for convolutional neural networks (CNNs). Yet, existing methods have two major limitations: (1) requiring re-training to maintain accuracy for *deep* CNNs, and (2) needing 16-bit floating-point or 8-bit fixed-point for a good accuracy. In this paper, we propose a low precision (8-bit) floating-point (LPFP) quantization method for FPGA-based acceleration to overcome the above limitations. Without any re-training, LPFP finds an optimal 8-bit data representation with negligible top-1/top-5 accuracy loss (within 0.5%/0.3% in our experiments, respectively, and significantly better than existing methods for *deep* CNNs). Furthermore, we implement one 8-bit LPFP multiplication by one 4-bit multiply-adder (MAC) and one 3-bit adder, and therefore implement *four* 8-bit LPFP multiplications using one DSP slice of Xilinx Kintex 7 family (KC705 in this paper) while one DSP can implement only *two* 8-bit fixed-point multiplications. Experiments on six typical CNNs for inference show that on average, we improve throughput by $64.5\times$ over Intel i9 CPU and by $1.5\times$ over existing FPGA accelerators. Particularly for VGG16 and YOLO, compared to six recent FPGA accelerators, we improve average throughput by $3.5\times$ and $27.5\times$ and improve average throughput per DSP by $4.1\times$ and $5\times$, respectively. To the best of our knowledge, this is the first in-depth study to simplify one multiplication for CNN inference to one 4-bit MAC and implement four multiplications within one DSP while maintaining comparable accuracy without any re-training.

Index Terms—low precision floating-point, CNN, deep learning, FPGA processor, FPGA acceleration

I. INTRODUCTION

CONVOLUTIONAL neural networks (CNNs) have demonstrated a breakthrough in performance for a broad range of applications including object recognition [1], object detection [2] and speech recognition [3]. However, CNNs often have huge computation complexity. This motivates accelerating CNNs by CPU/GPU clusters [4], FPGAs [5] and ASICs [6]. Customized accelerators/processors on FPGAs have shown more promising throughput and power efficiency than traditional CPU/GPU clusters [7], [8].

Larger and deeper CNNs have been developed to improve performance for a broader range of scenarios. For example, the top-5 error for ImageNet [9] classification decreases from 17% to 2.9%. However, computation complexity and number of parameters increase dramatically as depicted in Figure 1. To be specific, the computation complexity of a feed-forward process of a 224×224 RGB image increases from

C. Wu, M. Wang, X. Chu, K. Wang and L. He are with the Department of Electrical and Computer Engineering, University of California, Los Angeles, CA, 90095, USA (e-mail: chenwu1989@ucla.edu, mingyuw@ucla.edu, xinyuansjtu@gmail.com, wangk@ucla.edu, lhe@ee.ucla.edu).

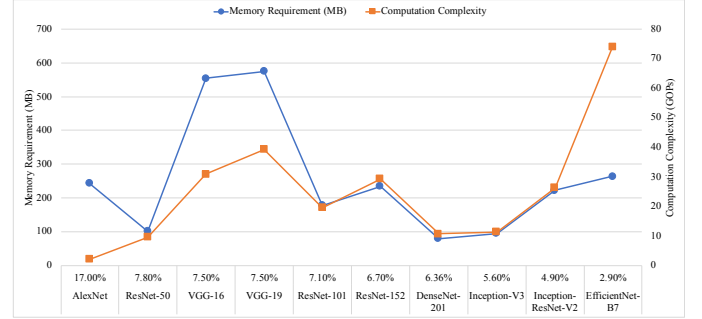


Fig. 1. Computation complexity and memory requirement with respect to different CNNs.

TABLE I
RESOURCE UTILIZATION OF MULTIPLIERS ON FPGA FOR DIFFERENT DATA REPRESENTATIONS. DSP: DIGITAL SIGNAL PROCESSING, LUT: LOOK-UP TABLE, FF: FLIP-FLOP. *M4E3*: 1-BIT SIGN, 4-BIT MANTISSA AND 3-BIT EXPONENT.

Data Representation	DSP	LUT	FF
one 16-bit floating multiplication	1	85	167
one 16-bit fixed multiplication	1	0	0
two 8-bit fixed multiplications	1	2	0
four 8-bit floating (<i>M4E3</i>) multiplications	1	20	27

2.27 GOP of AlexNet [10] in 2012 to 74 GOP of EfficientNet-B7 [11] in 2019. At the same time, the number of parameters stays large at 264 MB. Such great computation complexity makes it harder for general-purpose processor to meet the requirements of real-time applications. On the other hand, the great quantities of parameters lead to a big challenge for communication between off-chip and on-chip memories because of bandwidth constraints.

There are two types of research to reduce computation and parameter complexities for CNN inference. The first one is deep compression including weight pruning, weight quantization and compression storage [12], [13]. However, irregularity caused by deep compression degrades parallelism and hardware performance. Cambricon-S [14] alleviates irregularity in sparse neural networks through a software/hardware co-design approach to improve hardware performance. However, all the above accelerators need time-consuming re-training process to maintain accuracy.

The second type of research is more efficient data representation, also known as quantization for circuit implementation. [15] used 16-bit floating-point in contrast to 32-bit commonly used for computing. However, one 16-bit floating-point multiplier on FPGA needs 1 DSP, 85 LUTs and 167 FFs when using Xilinx floating-point IP [16] as shown in